

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/277082487>

CPU Efficiency Of Multicore JVM Paper ID PDP4427

Data · May 2015

CITATIONS

0

READS

156

3 authors:



Khondker Shajadul Hasan
University of Houston - Clear Lake

46 PUBLICATIONS 211 CITATIONS

SEE PROFILE



Nicolas G. Grounds
University of Oklahoma

10 PUBLICATIONS 47 CITATIONS

SEE PROFILE



John Antonio
University of Oklahoma

102 PUBLICATIONS 1,016 CITATIONS

SEE PROFILE

Predicting CPU Availability of a Multi-core Processor Executing Concurrent Java Threads

Khondker Shajadul Hasan¹, Nicolas G. Grounds², John K. Antonio¹

¹School of Computer Science, University of Oklahoma, 110 W. Boyd St., Norman, OK 73019, USA
shajadul@ou.edu, antonio@ou.edu

²MSCI, 201 David L Boren Blvd # 300, Norman, OK 73072, USA
nicolas.grounds@msci.com

Abstract - Techniques for predicting the availability of CPU resources associated with the execution of multiple concurrent Java threads on a multi-core architecture are introduced. Prediction of CPU availability is important in the context of making thread assignment and scheduling decisions. Theoretically derived upper and lower bound formulas for estimating CPU availability are introduced. Input parameters to the formulas include: number of cores; number of threads; and the unloaded CPU usage factor for each thread. Extensive experimental studies and statistical analysis are performed to validate the theoretical bounds and provide a basis for an empirical model for predicting CPU availability. To facilitate scientific and controlled empirical evaluation, synthetically generated threads are employed that are parameterized by their unloaded CPU usage factor, defined as the fraction of time a thread spends utilizing CPU resources on an unloaded system.

Keywords: Concurrent threads; CPU availability; Multi-core processors; Java Virtual Machine.

1. Introduction

Multithreading is a common technique used for exploiting performance from multi-core processors. When the number of threads assigned to a multi-core processor is less than or equal to the number of CPU cores associated with the processor, then the performance of the CPU is predictable, and is often nearly ideal. When the number of assigned threads is more than the number of CPU cores, the resulting CPU performance can be more difficult to predict. For example, assigning two CPU-bound threads to a single core results in CPU availability of about 50%, meaning that roughly 50% of the CPU resource is available for executing either thread. Alternatively, if two I/O-bound threads are assigned to a single core, it is possible that the resulting CPU availability is nearly 100%, provided that the usage of the CPU resource by each thread is fortuitously interleaved. However, if the points in time where both I/O-bound threads do require the CPU

resource overlap (i.e., they are not interleaved), then it is possible (although perhaps not likely) that the CPU availability of the two I/O bound threads could be as low as 50%. Predicting the availability of CPU resources when the number of threads assigned to the processor exceeds the number of cores is important in making thread assignment and scheduling decisions. Precise values of CPU availability are difficult to predict because of a dependence on many factors, including context switching overhead, CPU usage requirements of the threads, the degree of interleaving of the timing of the CPU requirements of the threads, and the characteristics of the thread scheduler of the underlying operating system (or Java Virtual Machine). Due to the complex nature of the execution environment, an empirical approach is employed to evaluate proposed CPU availability prediction models and formulas.

The success of approaches for assigning threads (or processes) to multi-core systems relies on the existence of reasonably accurate models for estimating CPU availability. This is because there is a strong relationship between a thread's total execution time and the availability of CPU resources used for its execution. Therefore, predicting the CPU availability that results when threads are assigned to a processor is a basic problem that arises in many important contexts [1].

In the present paper, an extensive collection of empirical measurements taken from both single- and multi-core processors provide a basis for validating proposed analytical models for estimating CPU availability. The proposed models are theoretically derived upper and lower bound formulas for CPU availability. Confidence interval and moving average statistics from measured CPU availability (from different empirical case studies) validate the utility of these theoretical models. Case studies for a single core machine involve spawning 2, 3 and 4 concurrent threads with randomly selected CPU usage factors. Each case study includes a collection of about 2,000 sample executions. For a quad core machine, similar

case studies are conducted involving 8, 12 and 16 concurrent threads, again with randomly selected CPU usage factors for the threads.

An interesting outcome of the empirical case studies is that the variation in measured CPU availability is very low whenever the sum of the threads' CPU usage factors is either relatively low, or relatively high. Thus, prediction of CPU availability is quite accurate when the CPU is either lightly or heavily loaded. When the total CPU loading (i.e., sum of all threads' CPU usage factors) is moderate, the realized availability of CPU resources has more variation and thus is less predictable. However, even the largest variation in CPU availability measured was typically no more than 20%, with a 90% confidence. The empirical studies also show that, as the number of concurrent threads increases, the context switching overhead degrades the performance of thread execution; resulting in a slightly wider gap between the theoretically derived upper bound and measured availability values. In both single- and multi-core cases, when the number of threads is above the number of CPU cores, the performance of thread execution is predicted reasonably well by the theoretical upper bound formula for CPU availability.

The rest of the paper is organized in the following manner. Section 2 discusses relevant background related to the execution of Java threads, and motivates the importance of predicting CPU availability. Section 3 introduces the specific thread execution model assumed in this paper; from this model, theoretical derivations of upper and lower bound formulas for CPU availability are provided. Section 4 presents the empirical studies including benchmarking, case study measurements for single- and multi-core CPUs, and statistical analysis of the results. Finally, Section 5 contains concluding remarks and suggestions for future work.

2. Execution of Concurrent Java Threads

The primary focus of this paper is to estimate the CPU resource availability of a Java Virtual Machine (JVM) executing concurrent threads. The concept of a Java monitor [2] is useful for describing how threads are executed by a JVM. A graphical depiction of such a monitor is shown in Figure 1, which contains three major sections. In the center, the large circular area represents the owner, which contains two active threads, represented by the small shaded circles. The number of threads the owner can contain (concurrently) is bounded by the number of CPU cores. At the upper left, the rectangular area represents the entry set. At the lower right, the other rectangular area represents the wait set. Waiting or suspended threads are illustrated as striped circles.

Figure 1 also shows several labeled “doors” that threads must pass through to interact with the monitor. In the figure, one thread is suspended in the entry set and one thread is suspended in the wait set. These threads will remain where they are until one of the active threads releases its position in the monitor. An active thread can release the monitor in either of two ways: it can complete the monitor region it is executing or it can execute a wait command. If it completes the monitor region, it exits the monitor via the door labeled E. If it executes a wait command, it releases the monitor and passes through door labeled C, the door into the wait set.

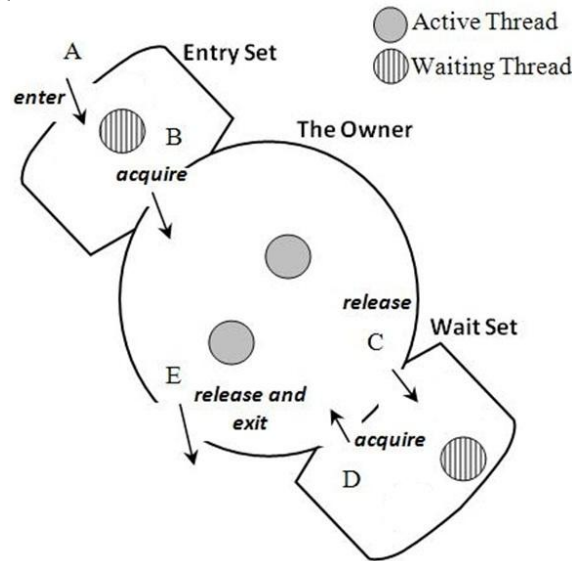


Figure 1: A typical Java Monitor, derived from [2].

To illustrate issues associated with predicting CPU availability, assume the monitor of Figure 1 is associated with a dual-core JVM. From the figure, there are a total of four threads assigned to the JVM; two are currently in a waiting state and two are currently active. If all four threads are CPU-bound, meaning that their unloaded CPU usage factor is 100%, then it is clear that the availability of the CPU resources (i.e., both CPU cores) will be about 50%. If, on the other hand, some threads are I/O-bound (e.g., having CPU usage factors less than say 25%) then predicting CPU availability is not as straightforward. In general, the realized CPU availability depends upon the scheduling scheme employed by the JVM (and/or the underlying OS) for transitioning threads between the active and waiting states.

In the next section, an analytical framework is developed for estimating CPU availability associated with executing concurrent threads on a multi-core JVM. The primary contribution of the section is the derivation of upper and lower bound formulas for CPU availability.

3. Bounds for Multi-core CPU Availability

In this section, an analytical framework is developed for estimating CPU availability associated with executing concurrent threads on a multi-core processor. The primary contribution of this section is the derivation of upper and lower bound formulas for CPU availability. A thread is modeled by a series of alternating work and sleep phases. For the purposes of this study, the work portion of a phase is CPU-bound and requires a fixed amount of computational work (i.e., CPU cycles). The sleep portion of a phase does not consume CPU cycles, and its length relative to that of the work portion is used to define the CPU load usage factor for a thread. Figure 2 shows three work-sleep phases of a thread.

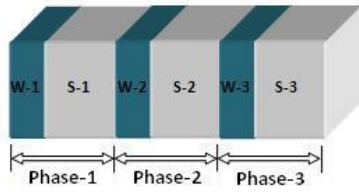


Figure 2: Three work-sleep phases of a thread.

In the assumed framework, a thread in the work portion of a phase will remain in the work portion until it has consumed enough CPU cycles to complete the allotted work of that portion. After completing the work portion of a phase, the thread then enters in the sleep portion where it sleeps (does not consume CPU cycles) for an amount of time defined by the CPU usage factor. When multiple threads are spawned concurrently, the JVM runs those threads employing a time sharing technique (refer to the discussion of the Java Monitor in Section 2). The CPU availability (and performance) will be degraded when the work phase of all threads overlap each other in time. Figure 3 depicts a scenario where 3 threads with identical work-sleep phases are executed in a single-core execution environment.

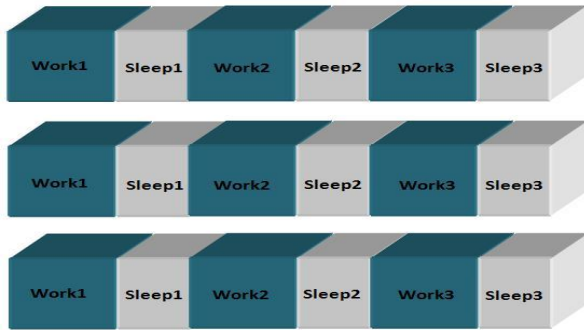


Figure 3: Three concurrent threads having identical work-sleep phases and CPU usage factors executing on a single-core environment.

Each thread gets a maximum of 1/3 of the available CPU resource during the work portions of their phases, resulting in 3-times wider work portion execution time than would be the case for a single thread scenario. Alternatively, if the work portions of these three threads are staggered to where there is no overlap, then there is no contention for the CPU resource and the CPU availability is essentially 100%, as shown in Figure 4. That is, all the work phases of concurrent threads are separated in time so that each thread can get the full usage of the CPU the moment it is first needed by each thread. The ideal phasing illustrated in Figure 4 requires that the work of any two threads can be accomplished within the time of one sleep portion of a phase.

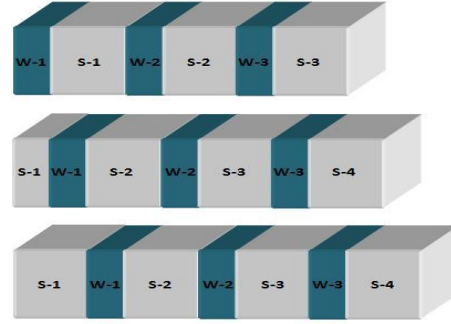


Figure 4: Three concurrent threads with identical work-sleep phases and CPU usage factors with non-overlapping work portions.

In the proposed framework, each thread is parameterized by a CPU usage factor and a total amount of computational work to accomplish. The CPU usage factor is defined as the time required to complete the work portion of a work-sleep phase on an unloaded CPU, divided by the total time of a work-sleep phase. A thread having zero sleep length has a CPU usage factor of 100%, which is also called a CPU-bound thread.

For a single core machine, the following formulas define upper and lower bounds for CPU availability. Here, n is the number of threads assigned to the single core machine and L is the aggregate loading factor, defined as the sum of CPU usage factors of all n threads. The upper bound formula for CPU availability is:

$$\bar{a} = \frac{1}{\text{Max}(1, L)} = \begin{cases} 1, & \text{if } L < 1 \\ 1/L & \text{if } L \geq 1 \end{cases} \quad (1)$$

and the lower bound formula is:

$$\underline{a} = \frac{1}{1 + \left(\frac{n-1}{n}\right) \times L} \quad (2)$$

The upper bound formula represents the best case CPU availability, which is illustrated by the example of Figure 4 in which none of the threads use the CPU

resource concurrently. Provided that the sum of the usage factors of the threads is less than unity, then it is possible that the CPU availability could be as high as unity (i.e., 100%). When the sum of the CPU usage factors is greater than unity, then the best possible value for CPU availability is $1/L$. The lower bound formula is associated with a situation in which the threads' usage of the CPU resource has maximum overlap, as depicted in Figure 3 in which all threads always use the CPU resource concurrently.

For a multi-core machine with c cores, the following formulas define upper and lower bounds for CPU availability. The upper bound formula is:

$$\bar{a} = \frac{1}{\text{Max}(1, L/c)} = \begin{cases} 1, & \text{if } L/c < 1 \\ c/L & \text{if } L/c \geq 1 \end{cases} \quad (3)$$

and the lower bound is:

$$\underline{a} = \frac{1}{1 + \left(\frac{n-1}{n}\right) \times \left(\frac{L}{c}\right)} \quad (4)$$

Note that Eqs. 3 and 4 are generalizations of Eqs. 1 and 2, i.e., for the case $c = 1$, Eqs. 3 and 4 are identical to Eqs. 1 and 2.

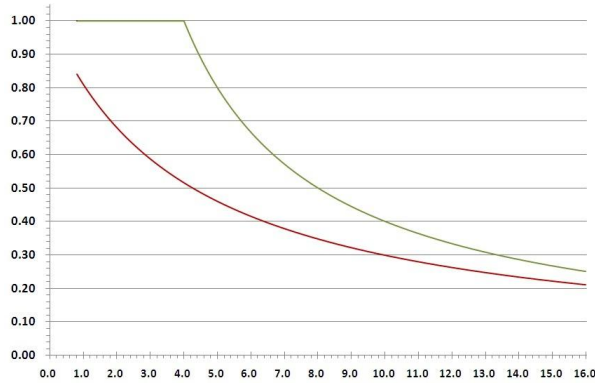


Figure 5: Upper and lower bounds for CPU availability prediction for $c = 4$ cores.

Figure 5 shows plots of the upper and lower bound formulas for the case $c = 4$. Note that the difference in upper and lower bounds can be significant for moderate values of aggregate loading. In the following section, experimental studies are performed to determine actual measured values of CPU availability in relation to these bounds.

4. Experimental Studies

4.1 Overview

The purpose of the experimental studies is to empirically measure CPU availability as a function of aggregate loading for collections of threads with

randomly selected CPU usage factors. For the study, threads are generated synthetically so that their CPU usage factors can be set accurately. The measured CPU availability associated with a collection of threads executing on a processor is defined by the ratio between the ideal time required to execute one of the threads on an unloaded processor divided by that thread's execution time on a loaded processor. About 2,000 randomly selected collections of threads are generated for each study; each randomly generated collection of threads provides one measurement of CPU availability. The major part of the experimental system's flow control is shown in Figure 6.

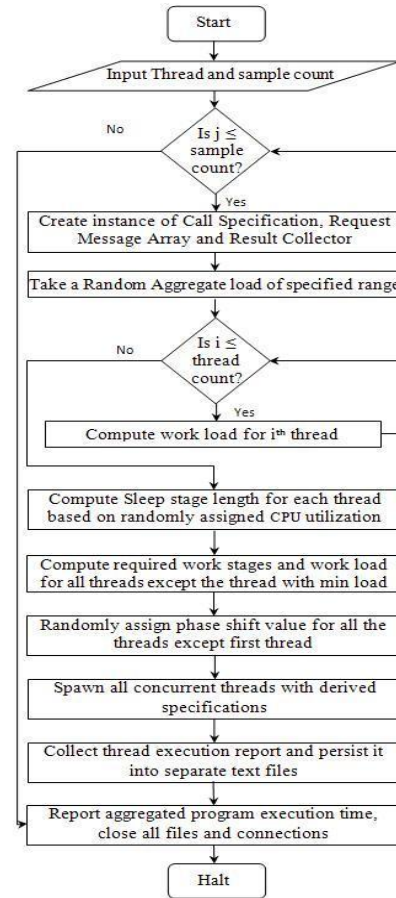


Figure 6: Flow chart for load distribution and execution process of threads.

To ensure a uniform sampling of data across the values of possible aggregate loadings, a random value of aggregate loading is chosen first. For example, for a scenario having two threads, a value of aggregate loading is chosen between a small value ϵ and 2; denote this value as L . Then a random value is chosen between $\max\{\epsilon, L-1\}$ and $\min\{1, L\}$, which defines the CPU usage factor of the first thread, say T_1 ; the CPU usage factor of the second thread is then

defined as $T_2 = L - T_1$. In general, for n threads, the following algorithm is used to randomly assign the CPU usage factors for a given value of aggregate loading L :

```

For  $i = 1$  to  $n - 1$  do
   $LB = \text{Max}((L - \sum_{j=1}^{i-1} T_j - (n - i)), \epsilon)$ 
   $UB = \text{Min}((L - \sum_{j=1}^{i-1} T_j - (n - i) * \epsilon), 1)$ 
  Pick  $T_i$  from the following range
   $T_i \in [LB, UB]$ 
   $\Delta = UB - LB$ 
   $T_i = \Delta \times T_i + LB$ 
End
 $T_n \leftarrow L - \sum_{i=1}^{n-1} T_i$ 

```

Recall that the CPU usage factor of a thread defines the relative amount of work performed during its work phase relative to its sleep phase. As described in the flow chart of Figure 6, a random phase shift is also assigned to each thread. The range of the phase shift value is from 0 to length of the phase. This random phase shift provides a degree of staggering of phases among the threads. Figure 4 is an example of the best case staggering; whereas Figure 3 is an example of worst case staggering in terms of CPU availability.

4.2 Experimental Environment

The system used for evaluating the single core test cases is an Intel Xeon CPU E5540 @ 2.53GHz clock speed, 1,333 MHz bus speed and 4 GB of RAM. The system used for evaluating the multi-core test cases is an Intel Core 2 Quad (quad-core processor), 2.83GHz clock speed, 1,333MHz bus speed with 4 GB of RAM. Due to the different configurations of the single- and quad-core systems, benchmarking for the single-core and quad-core machines were determined separately. The JVM used for these experiments is JDK 1.6. Threads deployed here are independent tasks, meaning there are no interdependencies among threads such as message passing. Threads are spawned concurrently with workloads and phase shifts as described in the previous section. When a collection of threads completes, a report of the threads' execution is produced, which contains start time, work time, sleep time, number of phases, and end time.

4.3 Benchmarking

Benchmarking an unloaded system enables the calculation of parameters associated with the work and sleep portions of the phases to synthesize a particular CPU usage factor. For setting up the benchmarks, each thread was assigned 2.0×10^8 units of synthetic CPU work. Threads need to accomplish this total work in 50 work-sleep phases. In the work component of each phase, these threads accomplish 4.0×10^6 units of

work. For calculating the CPU usage factors (5%, 10%, 15%, etc.), the sleep phase length is varied using the following equation.

$$\text{sleep} = \left(\frac{\text{stage work time} \times (1 - \text{CPU usage})}{\text{CPU usage}} \right) \quad (5)$$

According to the formula above, the sleep time increases as the CPU usage decreases. Thus, a thread with low CPU usage will sleep longer than other threads having a higher CPU usage factors.

4.4 Empirical CPU Availability Case Studies

For measuring the CPU availability of the single-core processor, three case studies were conducted in which multiple (2, 3, and 4) threads were spawned concurrently. An aggregate CPU load L was selected randomly, and distributed among the threads as described in Section 4.1.

Figures 7 and 8 show measured CPU availability scatter graphs for 2 and 4 concurrent threads executing on the single-core processor, superimposed with the plots of the upper and lower bound formulas derived in Section 3. In these figures, the horizontal axis represents aggregated CPU load and the vertical axis represents CPU availability. Each small dot in these graphs is an independent test case measurement of CPU availability. There are 2,000 dots in each figure representing measured CPU availability value among the concurrent threads. A moving average line is also drawn through the data on the graphs for helping to visualize the average measured performance. A window size of 0.10 aggregate CPU load and incremental value of 0.01 was used to calculate the moving average values. A similar sliding window approach was employed to calculate the 90% confidence interval upper and lower limits.

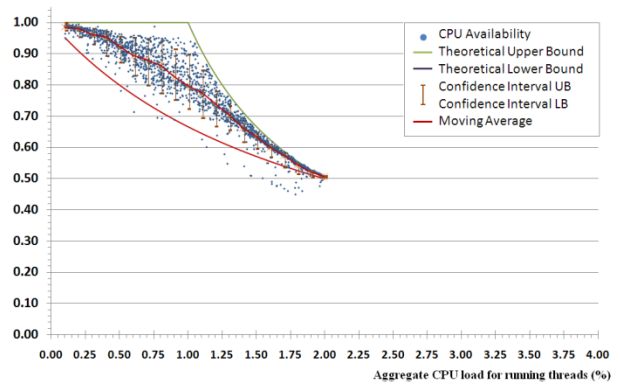


Figure 7: CPU availability of 2 threads in a single-core machine. Results of 2,000 independent test cases and 90% confidence intervals.

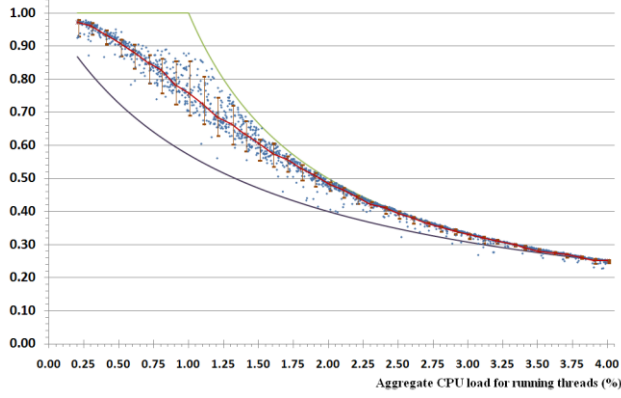


Figure 8: CPU availability of 4 threads in a single-core machine. Results of 2,000 independent test cases and 90% confidence intervals.

It is apparent from Figures 7 and 8 that the variation in CPU availability is low when the aggregate CPU loading is either relatively low or relatively high. Thus, CPU availability prediction is quite accurate when the CPU is either lightly or heavily loaded. When the total CPU loading is moderate, the measured CPU availability has more variation and thus it is less predictable. One of the intuitive reasons for low variation when threads have small CPU loading factors is their sleep phase lengths are wider, which decreases the probability of work portion overlap among the threads. On the other hand, when the aggregate CPU load is large, threads have smaller sleep phase and longer work phase lengths which almost always forces work phase to overlap and decrease performance, but in a predictable way.

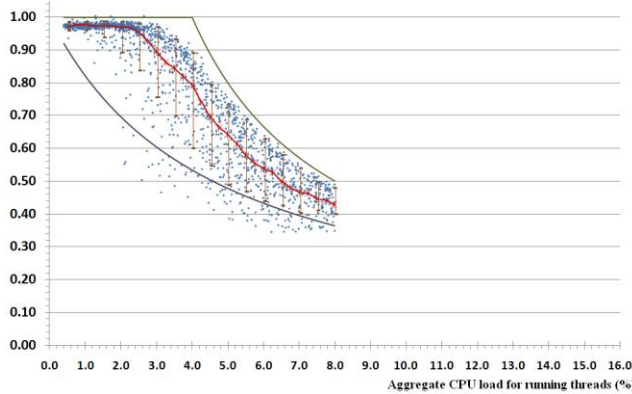


Figure 9: CPU availability of 8 threads in a quad-core machine. Results for 2,000 independent test cases and 90% confidence intervals.

For measuring the CPU availability of the quad-core processor, three case studies were conducted in which multiple (8, 12, and 16) threads were spawned concurrently to relatively compare the performance between single- and quad-core processors. A similar

approach has been employed to calculate the moving average and 90% confidence interval values. CPU availability graphs for 8 and 16 threads are shown in Figure 9 and Figure 10 respectively. Figure 9 shows a higher variation of CPU availability compared with Figure 7, which shows CPU availability of 2 threads in a single-core machine. During the thread execution life cycle, depending on CPU availability, threads might be allocated in different cores for load balancing which decrease the probability of work phase overlap.

The empirical results for the quad-core processor also show that the CPU availability prediction is quite accurate when the CPU is either lightly or heavily loaded. When the total CPU loading is moderate, the measured CPU availability has more variation and thus it is less predictable.

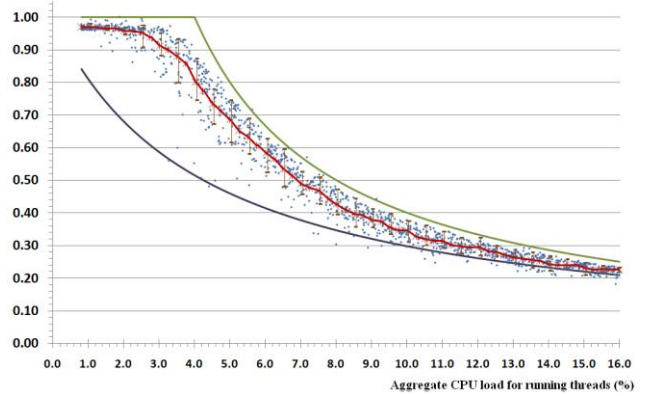


Figure 10: CPU availability of 16 threads in a quad-core machine. Results of 2,000 independent test cases and 90% confidence intervals.

The empirical results of the quad-core processor also shows that when the numbers of concurrent threads are exact multiples of the number of cores, it provides much better performance than when they are not. That measured variation can be as large as 10% of the overall CPU availability. From the empirical results of single- and multi-core processors, and measured CPU availability scatter graphs, it is apparent that theoretically derived upper and lower limits introduced in this paper bound actual measured values of CPU availability reasonably well.

In further reporting the results of the studies, it is convenient to define the normalized aggregate load, L/n , which is the aggregate load L normalized by the number of threads n . For sample values of normalized aggregate load, Table 1 shows the average measured CPU availability (Avg.), the difference in the upper and lower bound formulas (Bnd Diff) and the difference in the 90% confidence interval limits (CI Diff) for 2, 3 and 4 concurrent threads on the single-core processor.

Table 1: Average measured CPU availability (Avg.) for threads in a single-core machine along with differences of bound formulas (Bnd Diff) and 90% confidence interval limits (CI Diff).

L/n	2 Thread CPU Availability			3 Thread CPU Avail.			4 Thread CPU Availability		
	Avg	Bnd Diff	CI Diff	Avg	Bnd Diff	CI Diff	Avg	Bnd Diff	CI Diff
0.05	0.985	0.048	0.019	0.973	0.094	0.023	0.971	0.133	0.050
0.10	0.980	0.091	0.033	0.946	0.167	0.051	0.935	0.232	0.042
0.20	0.953	0.167	0.076	0.873	0.287	0.087	0.828	0.375	0.104
0.30	0.892	0.231	0.121	0.790	0.376	0.149	0.685	0.306	0.117
0.40	0.862	0.286	0.168	0.707	0.277	0.128	0.575	0.170	0.070
0.50	0.796	0.333	0.175	0.610	0.167	0.059	0.484	0.099	0.027
0.60	0.739	0.208	0.112	0.524	0.101	0.034	0.412	0.059	0.020
0.70	0.662	0.126	0.067	0.461	0.059	0.024	0.353	0.035	0.017
0.80	0.600	0.069	0.042	0.403	0.032	0.021	0.311	0.018	0.011
0.90	0.543	0.029	0.033	0.363	0.013	0.014	0.275	0.008	0.007
1.00	0.495	0.000	0.004	0.331	0.000	0.05	0.249	0.000	0.008

Table 1 shows that the difference between the upper and lower limits formula bounds can reach as high as 0.375, for 4 threads and a normalized aggregate loading of 0.20. However, the measured CPU availability is for this same case is much smaller, around 0.104. The difference of the formula-based bound is more precise when the CPU is lightly or heavily loaded.

Table 2: Average measured CPU availability (Avg.) for threads in a quad-core machine along with differences of bound formulas (Bnd Diff) and 90% confidence interval limits (CI Diff).

L/n	8 Thread CPU Availability			12 Thread CPU Avail.			16 Thread CPU Avail.		
	Avg	Bnd Diff	CI Diff	Avg	Bnd Diff	CI Diff	Avg	Bnd Diff	CI Diff
0.5	0.973	0.080	0.015	0.971	0.121	0.017	0.969	0.158	0.013
0.10	0.971	0.150	0.019	0.974	0.216	0.022	0.948	0.273	0.058
0.20	0.973	0.260	0.064	0.959	0.355	0.064	0.862	0.429	0.128
0.30	0.956	0.345	0.153	0.904	0.452	0.121	0.690	0.363	0.149
0.40	0.903	0.413	0.322	0.752	0.357	0.085	0.543	0.225	0.116
0.50	0.768	0.466	0.304	0.604	0.246	0.102	0.411	0.152	0.107
0.60	0.595	0.345	0.299	0.490	0.178	0.081	0.344	0.109	0.079
0.70	0.554	0.264	0.222	0.429	0.134	0.073	0.321	0.081	0.057
0.80	0.493	0.208	0.217	0.358	0.104	0.069	0.268	0.062	0.028
0.90	0.446	0.167	0.146	0.328	0.083	0.079	0.240	0.049	0.029
1.00	0.419	0.136	0.114	0.291	0.067	0.058	0.222	0.040	0.023

Similarly, for a quad core processor, Table 2 shows that the difference between the upper and lower limits formula bounds can reach as high as 0.429, for 16 threads and a normalized aggregate loading of 0.20. However, the measured CPU availability is for this same case is much smaller, around 0.128. The difference of the formula-based bound is more precise when the CPU is lightly or heavily loaded for quad-core processor as well.

5. Conclusion and Future work

This paper developed analytical models (and conducted empirical studies) for predicting (and measuring) CPU availability of JVMs supported by single- and multi-core architectures. As would be expected, degradation in CPU availability occurs when total CPU loading is greater than the total capacity of all CPU cores. In addition to total CPU loading, the total number of concurrent threads is a factor in predicting CPU efficiency; more threads generally incur more context switching overhead, which results in degraded availability. When the total load is less than the total capacity of all cores, the relative alignment of the working and sleeping phases of the threads can have a significant impact on CPU availability. Specifically, increased overlap of the work phases implies lower availability. It was demonstrated that shifting the relative phasing of the threads to reduce possible work phase overlap can improve the performance (i.e., CPU efficiency). Random aggregate load and phase shift values for concurrent threads were assigned for each for an extensive number of experimental measurements. A thread availability scatter plot provides a clear visualization of measured performance based on the density of the dots in the plot. These empirically measured availability values are showed to generally fall within theoretically derived upper and lower bound formulas. A 90% confidence interval for measured availability is shown to provide significantly tighter upper and lower limits than the theoretically derived formulas for upper and lower bounds.

6. References

- [1] Martha Beltrán, Antonio Guzmán and Jose Luis Bosque, "A new CPU Availability Prediction Model for Time-Shared Systems", *IEEE Computer*, Vol 57, July 2008.
- [2] Bill Venners, "Inside the Java 2 Virtual Machine", Thread Synchronization, URL: <http://www.artima.com/insidejvm/ed2/index.html>
- [3] Y. Zhang, W. Sun, and Y. Inoguchi, "Predicting running time of grid tasks on cpu load predictions", *Proceedings of the 7th IEEE/ACM International Conference on Grid Computing*, pp. 286–292, September 2006.
- [4] "Java thread performance", The behavior of Java threads under Linux NPTL, Amity Solutions Pty Ltd – Version 1.5, July 2, 2003, URL: http://www.amitysolutions.com.au/documents/NPTL_Java_threads.pdf
- [5] Ken Arnold and James Gosling, *The Java Programming Language*, Fourth Edition, Addison Wesley, 2005.
- [6] Vitaly Mikheev, "Switching JVMs May Help Reveal Issues in Multi-Threaded Apps", May 2010, <http://java.dzone.com/articles/case-study-switching-jvms-may>.
- [7] Analysis of Multithreaded Architecture for Parallel Computing, Rafael H. Saavedra, David E. Culler, Thorsten Eicken, *2nd Annual ACM Symposium on Parallel Algorithms and Architectures*, 1990.